

Contents

- **Interpolation**
 - **The interpolate operator**
 - **Semantics of symbolic interpolation**
 - **Interpolation operators**
 - **Adjoint interpolation**
 - **Interpolation across meshes**
 - **Interpolating onto sub-domain meshes**
 - **Interpolating onto other meshes**
 - **Interpolation from external data**
 - **Interpolation between mixed function spaces**
 - **Generating Functions with randomised values**

Interpolation

Firedrake offers highly flexible capabilities for interpolating expressions (functions of space) into finite element **Functions**. Interpolation is often used to set up initial conditions and/or boundary conditions. Mathematically, if $e(x)$ is a function of space and V is a finite element function space then $\text{interpolate}(e, V)$ is the **Function** $v_i \phi_i \in V$ such that:

$$v_i = \bar{\phi}_i^*(e)$$

where $\bar{\phi}_i^*$ is the i -th dual basis function to V suitably extended such that its domain encompasses e .

The extension of dual basis functions to e usually follows from the definition of the dual basis. For example, point evaluation and integral nodes can naturally be extended to any expression which is evaluatable at the relevant points, or integrable over that domain.

Firedrake will not impose any constraints on the expression to be interpolated beyond that its value shape matches that of the space into which it is interpolated. If the user interpolates an expression for which the nodes are not well defined (for example point evaluation at a discontinuity), the result is implementation-dependent.

The interpolate operator

The basic syntax for interpolation is:

```
# create a symbolic expression for the interpolation operation.
f_i = interpolate(expression, V)

# assemble the interpolation to get the result
f = assemble(f_i)
```

Here, the **interpolate()** function returned a **symbolic UFL Interpolate** expression. To calculate a concrete numerical result, we need to call **assemble()** on this expression.

It is also possible to interpolate an expression directly into an existing **Function**:

```
f = Function(V)
f.interpolate(expression)
```

This is a numerical operation, equivalent to:

```
f = Function(V)
f.assign(assemble(interpolate(expression, V)))
```

The source expression can be any **UFL** expression with the correct shape. UFL produces clear error messages in case of syntax or type errors, yet UFL expressions have good run-time performance, since they are translated to C interpolation kernels using **TSFC** technology. Moreover, UFL offers a rich language for describing expressions, including:

- The coordinates: in physical space as **SpatialCoordinate**, and in reference space as **ufl.geometry.CellCoordinate**.
- Firedrake **Functions**, derivatives of **Functions**, and **Constants**.
- Literal numbers, basic arithmetic operations, and also mathematical functions such as `sin`, `cos`, `sqrt`, `abs`, etc.
- Conditional expressions using UFL **conditional**.
- Compound expressions involving any of the above.

Here is an example demonstrating some of these features:

```
# g is a vector-valued Function, e.g. on an H(div) function space
f = assemble(interpolate(sqrt(3.2 * div(g)), V))
```

This also works when interpolating into a space defined on the facets of the mesh:

```
trace = FunctionSpace(mesh, "HDiv Trace", 0)
f = assemble(interpolate(expression, trace))
```

Interpolation is supported into most, but not all, of the elements that Firedrake provides. In particular it is not currently possible to interpolate into spaces defined by higher-continuity elements such as Argyris and Hermite.

Semantics of symbolic interpolation

Let U and V be finite element spaces with DoFs $\{\psi_i^*\}$ and $\{\phi_i^*\}$ and basis functions $\{\psi_i\}$ and $\{\phi_i\}$, respectively. The interpolation operator between U and V is defined

$$\begin{aligned}\mathcal{I}_V : U &\rightarrow V \\ \mathcal{I}_V(u)(x) &= \phi_i^*(u)\phi_i(x).\end{aligned}$$

We define the following bilinear form

$$\begin{aligned}I : U \times V^* &\rightarrow \mathbb{R} \\ I(u, v^*) &= v^*(u)\end{aligned}$$

where $v^* \in V^*$ is a linear functional in the dual space to V , extended so that it can act on functions in U . If we choose $v^* = \phi_i^*$ then $I(u, \phi_i^*) = \phi_i^*(u)$ gives the coefficients of the interpolation of u into V . This allows us to represent the interpolation as a form in **UFL**. This is exactly the **Interpolate** UFL object. Note that this differs from typical bilinear forms since one of the arguments is in a dual space. For more information on dual spaces in Firedrake, see [the relevant section of the manual](#).

Interpolation operators

2-forms are assembled into matrices, and we can do the same with the interpolation form. If we let u be a `TrialFunction(U)` (i.e. an argument in slot 1) and v^* be a `TestFunction(V.dual())` (i.e. a **Coargument** in slot 0) then

$$I(u, v^*) = I(\psi_j, \phi_i^*) = \phi_i^*(\psi_j) =: A_{ij}$$

The matrix A is the interpolation matrix from U to V . In Firedrake, we can assemble this matrix by doing

```
A = assemble(interpolate(TrialFunction(U), V))
```

Passing a **FunctionSpace** into the dual slot of **interpolate()** is syntactic sugar for `TestFunction(V.dual())`.

If $g \in U$ is a **Function**, then we can write it as $g = g_j\psi_j$ for some coefficients g_j . Interpolating g into V gives

$$I(g, v^*) = \phi_i^*(g_j\psi_j) = A_{ij}g_j,$$

so we can multiply the vector of coefficients of g by the interpolation matrix to obtain the coefficients of the interpolated function. In Firedrake, we can do this by

```
h = assemble(A @ g)
```

h is a **Function** in V representing the interpolation of g into V .

When interpolating a **Function** directly, for example

```
assemble(interpolate(Function(U), V))
```

Firedrake does not explicitly assemble the interpolation matrix. Instead, the interpolation is performed matrix-free.

Adjoint interpolation

The adjoint of the interpolation operator is defined as

$$\mathcal{I}_V^* : V^* \rightarrow U^*.$$

This operator interpolates **Cofunctions** in the dual space V^* into the dual space U^* . The associated form is

$$I^* : V^* \times U \rightarrow \mathbb{R}.$$

So to obtain the adjoint interpolation operator, we swap the arguments of the **Interpolate** form. In Firedrake, we can accomplish this in two ways. The first is to swap the argument numbers to the form:

```
Istar1 = interpolate(TestFunction(U), TrialFunction(V.dual()))
```

The second way is to use UFL's **adjoint()** operator, which takes a form and returns its adjoint:

```
Istar2 = adjoint(interpolate(TrialFunction(U), V))
```

If g^* is a **Cofunction** in V^* then we can interpolate it into U^* by doing

$$I^*(g^*, u) = g_i^* \phi_i^*(\psi_j) = g_i^* A_{ij}.$$

This is the product of the adjoint interpolation matrix A^* and the coefficients of g^* . In Firedrake, we can do this by

```
cofunc = assemble(inner(1, TestFunction(V)) * dx) # a cofunction in V*
res1 = assemble(interpolate(TestFunction(U), cofunc)) # a cofunction in U*
```

Again, Firedrake does not explicitly assemble the adjoint interpolation matrix, but performs the interpolation matrix-free. To perform the interpolation with the assembled adjoint interpolation operator, we can take the **action()** of the operator on the **Cofunction**:

```
res2 = assemble(action(Istar1, cofunc)) # same as res1
```

The final case is when we interpolate a **Function** into **Cofunction**:

```
interpolate(u, cofunc)
```

This interpolation has zero arguments and hence is assembled into a number. Mathematically, we have

$$I^*(g^*, u) = g_i^* \phi_i^*(u_j \psi_j) = g_i^* A_{ij} u_j.$$

which indeed contracts into a number.

Interpolation across meshes

The interpolation API supports interpolation across meshes where the target function space has any finite element which supports interpolation, as specified in the list of **supported elements**. Vector, tensor, and mixed function spaces can also be interpolated into from other meshes as long as they are constructed from these spaces.

The list of supported elements above is only for *target* function spaces. Function spaces on the *source* mesh can be built from most of the supported elements.

There are few constraints on the meshes involved: the target mesh can have a different cell shape, topological dimension, or resolution to the

source mesh. There are many use cases for this: For example, two solutions to the same problem calculated on meshes with different resolutions or cell shapes can be interpolated onto one another, or onto a third, finer mesh, and be directly compared.

Interpolating onto sub-domain meshes

The target mesh for a cross-mesh interpolation need not cover the full domain of the source mesh. Volume, surface and line integrals can therefore be calculated by interpolating onto the mesh or **immersed manifold** which defines the volume, surface or line of interest in the domain. The integral itself is calculated by calling **assemble()** on an appropriate form over the target mesh function space:

```

from firedrake.mesh import Mesh, plex_from_cell_list
# Start with a simple field exactly represented in the function space over
# the unit square domain.
m = UnitSquareMesh(2, 2)
V = FunctionSpace(m, "CG", 2)
x, y = SpatialCoordinate(m)
f = Function(V).interpolate(x * y)

# We create a 1D mesh immersed 2D from (0, 0) to (1, 1) which we call "line".
# Note that it only has 1 cell
cells = np.asarray([[0, 1]])
vertex_coords = np.asarray([[0.0, 0.0], [1.0, 1.0]])
plex = plex_from_cell_list(1, cells, vertex_coords, comm=m.comm)
line = Mesh(plex, dim=2)

# We want to calculate the line integral of f along it. To do this we
# create a function space on the line mesh...
V_line = FunctionSpace(line, "CG", 2)

# ... and interpolate our function f onto it.
f_line = assemble(interpolate(f, V_line))

# The integral of f along the line is then a simple form expression which
# we assemble:
assemble(f_line * dx) # this outputs sqrt(2) / 3

```

For more on forms, see [this section of the manual](#).

Interpolating onto other meshes

If the target mesh extends outside the source mesh domain, then cross-mesh interpolation will raise a **DofNotDefinedError**.

```

# These meshes only share some of their domain
src_mesh = UnitSquareMesh(2, 2)
dest_mesh = UnitSquareMesh(3, 3, quadrilateral=True)
dest_mesh.coordinates.dat.data_wo[:] *= 2

# We consider a simple function on our source mesh...
x_src, y_src = SpatialCoordinate(src_mesh)
V_src = FunctionSpace(src_mesh, "CG", 2)
f_src = Function(V_src).interpolate(x_src**2 + y_src**2)

# ... and want to interpolate into a function space on our target mesh ...
V_dest = FunctionSpace(dest_mesh, "Q", 2)

# ... but get a DofNotDefinedError if we try
f_dest = assemble(interpolate(f_src, V_dest)) # raises DofNotDefinedError

```

This can be overridden with the optional `allow_missing_dofs` keyword argument:

```

# Setting the allow_missing_dofs keyword allows the interpolation to proceed.
f_dest = assemble(interpolate(f_src, V_dest, allow_missing_dofs=True))

f_dest = Function(V_dest).interpolate(f_src, allow_missing_dofs=True)

```

In this case, the missing degrees of freedom (DoFs, the global basis function coefficients which could not be set) are, by default, set to zero:

```
dest_eval15 = PointEvaluator(dest_mesh, [[1.5, 1.5]])
dest_eval15.evaluate(f_dest) # returns 0.0
```

If we specify an output **Function** then the missing DoFs are unmodified.

We can optionally specify a value to use for our missing DoFs. Here we set them to be nan ('not a number') for easy identification:

```
f_dest = assemble(interpolate(
    f_src, V_dest, allow_missing_dofs=True, default_missing_val=np.nan
))
dest_eval15.evaluate(f_dest) # returns np.nan
```

If we specify an output **Function**, this overwrites the missing DoFs.

If we don't set `default_missing_val` then any missing DoFs are left as they were prior to interpolation:

```
x_dest, y_dest = SpatialCoordinate(dest_mesh)
f_dest = Function(V_dest).interpolate(x_dest + y_dest)
dest_eval05.evaluate(f_dest) # returns x_dest + y_dest = 1.0
```

```
assemble(interpolate(f_src, V_dest, allow_missing_dofs=True), tensor=f_dest)
dest_eval05.evaluate(f_dest) # now returns x_src^2 + y_src^2 = 0.5
```

Similarly, using the `interpolate()` method on a **Function** will not overwrite the pre-existing values if `default_missing_val` is not set:

```
f_dest.interpolate(f_src, allow_missing_dofs=True)
```

Interpolation from external data

Unfortunately, UFL interpolation is not applicable if some of the source data is not yet available as a Firedrake **Function** or UFL expression. Here we describe a recipe for moving external data to Firedrake fields.

Let us assume that there is some function `mydata(X)` which takes as input an $n \times d$ array, where n is the number of points at which the data values are needed, and d is the geometric dimension of the mesh. `mydata(X)` shall return a n long vector of the scalar values evaluated at the points provided. (Assuming that the target **FunctionSpace** is scalar valued, although this recipe can be extended to vector or tensor valued fields.) Presumably `mydata` works by interpolating the external data source, but the precise details are not relevant now. In this case, interpolation into a target function space V proceeds as follows:

```
# First, grab the mesh.
m = V.mesh()

# Now make the VectorFunctionSpace corresponding to V.
W = VectorFunctionSpace(m, V.ufl_element())

# Next, interpolate the coordinates onto the nodes of W.
X = assemble(interpolate(m.coordinates, W))

# Make an output function.
f = Function(V)

# Use the external data function to interpolate the values of f.
f.dat.data[:] = mydata(X.dat.data_ro)
```

This will also work in parallel, as the interpolation will occur on each process, and Firedrake will take care of the halo updates before the next operation using `f`.

For interaction with external point data, see the [corresponding manual section](#).

Interpolation between mixed function spaces

Assembly of interpolation operators between mixed function spaces is also supported. Each component of the mixed space may be on different meshes. For example, consider the following mixed finite element spaces:

$$\begin{aligned} W &= V_1 \times V_2 \\ U &= V_3 \times V_4 \end{aligned}$$

where each V_i is a finite element space defined on possibly different meshes. We can assemble the interpolation matrix from U to W in Firedrake as follows:

```
interp = interpolate(TrialFunction(U), W)
I = assemble(interp, mat_type="nest")
```

We specified `mat_type="nest"` here to obtain a PETSc MatNest matrix, but Firedrake also supports assembly of `mat_type="aij"` and `mat_type="matfree"` interpolation matrices between mixed function spaces. In this example I is a block diagonal matrix, with each block given by

$$\begin{pmatrix} V_3 \rightarrow V_1 & 0 \\ 0 & V_4 \rightarrow V_2 \end{pmatrix}$$

The off-diagonal blocks are zero since the dofs are applied component-wise. Firedrake's form compiler recognises this and avoids assembling the zero blocks.

We can assemble more general interpolation matrices between mixed function spaces by interpolating vector expressions with arguments. For example, by doing

```
u0, u1 = TrialFunctions(U)
expr = as_vector([u0 + u1, u0 + u1])
interp = interpolate(expr, W)
I2 = assemble(interp, mat_type="nest")
```

we can assemble the interpolation matrix with block structure

$$\begin{pmatrix} V_3 \rightarrow V_1 & V_4 \rightarrow V_1 \\ V_3 \rightarrow V_2 & V_4 \rightarrow V_2 \end{pmatrix}$$

Here we obtain non-zero off-diagonal blocks by including both components of the trial function in each component of the expression.

Generating Functions with randomised values

The **randomfunctiongen** module wraps the external numpy package `numpy.random`, which gives Firedrake users an easy access to many stochastically sound random number generators, including **PCG64**, **Philox**, and **SFC64**, which are parallel-safe. All distribution methods defined in `numpy.random`, are made available, and one can pass a **FunctionSpace** to most of these methods to generate a randomised **Function**.

```
mesh = UnitSquareMesh(2,2)
V = FunctionSpace(mesh, "CG", 1)
# PCG64 random number generator
pcg = PCG64(seed=123456789)
rg = RandomGenerator(pcg)
# beta distribution
f_beta = rg.beta(V, 1.0, 2.0)

print(f_beta.dat.data)

# produces:
# [0.56462514 0.11585311 0.01247943 0.398984 0.19097059 0.5446709 0.1078666 0.2178807 0.64848515]
```
